

Methods

CSE160: Computer Science A: Honors

Paul Fodor

Stony Brook University

<http://www.cs.stonybrook.edu/~cse160>

Contents

- Motivation: Opening Problem
- Why write methods?
- Defining Methods:
 - Method Signature: modifiers,
 - Formal and Actual Parameters
 - Return Value Type
 - Calling Methods
- Call Stacks
- Benefits of Methods: Reuse, Method Abstraction and Information hiding
- Call-by-value
- Overloading
- CAUTION: all execution paths need returns (if a value is returned)

Motivation: Opening Problem

Find multiple sums of integers:

- from 1 to 10,
- from 20 to 30,
- from 35 to 45,
- ...

Opening Problem

- Repeat/copy code:

```
int sum = 0;  
for (int i = 1; i <= 10; i++)  
    sum += i;
```

```
System.out.println("Sum from 1 to 10 is " + sum);
```

```
sum = 0;  
for (int i = 20; i <= 30; i++)  
    sum += i;
```

```
System.out.println("Sum from 20 to 30 is " + sum);
```

```
sum = 0;  
for (int i = 35; i <= 45; i++)  
    sum += i;
```

```
System.out.println("Sum from 35 to 45 is " + sum);
```

Opening Problem Solution

- Use 1 method and invoke it multiple times!

```
public static int sum(int i1, int i2) {  
    int sum = 0;  
    for (int i = i1; i <= i2; i++)  
        sum += i;  
    return sum;  
}
```

```
public static void main(String[] args) {  
    System.out.println("Sum from 1 to 10 is " + sum(1, 10));  
    System.out.println("Sum from 20 to 30 is " + sum(20, 30));  
    System.out.println("Sum from 35 to 45 is " + sum(35, 45));  
}
```

Why write methods?

- To shorten your programs
 - avoid writing identical code twice or more
- To modularize your programs
 - fully tested methods can be trusted
- To make your programs more:
 - readable
 - reusable
 - testable
 - debugable

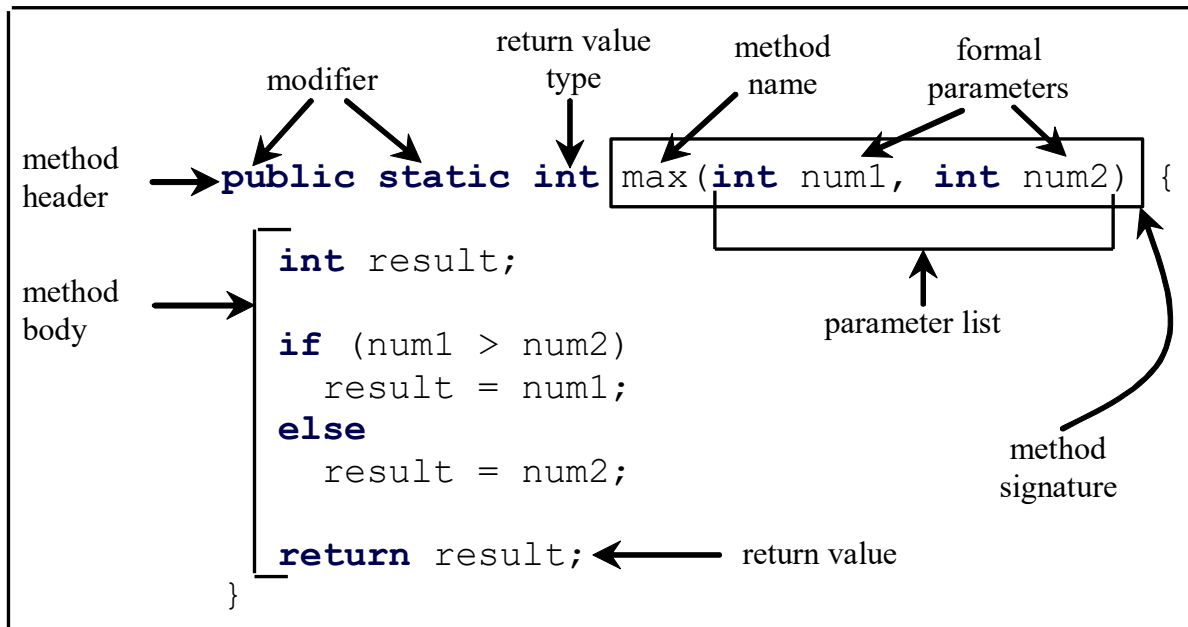
Rule of thumb

- If you have to perform some operation in more than in one place in your program, write a method to implement this operation and have other parts of the program use it

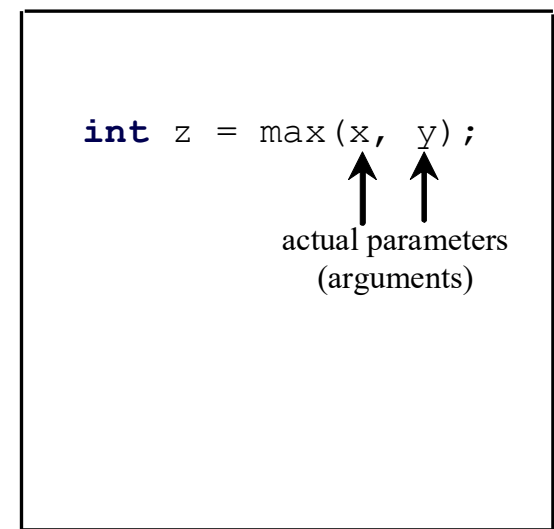
Defining Methods

- A *method* is a collection of statements that are grouped together to perform an operation.

Define a method



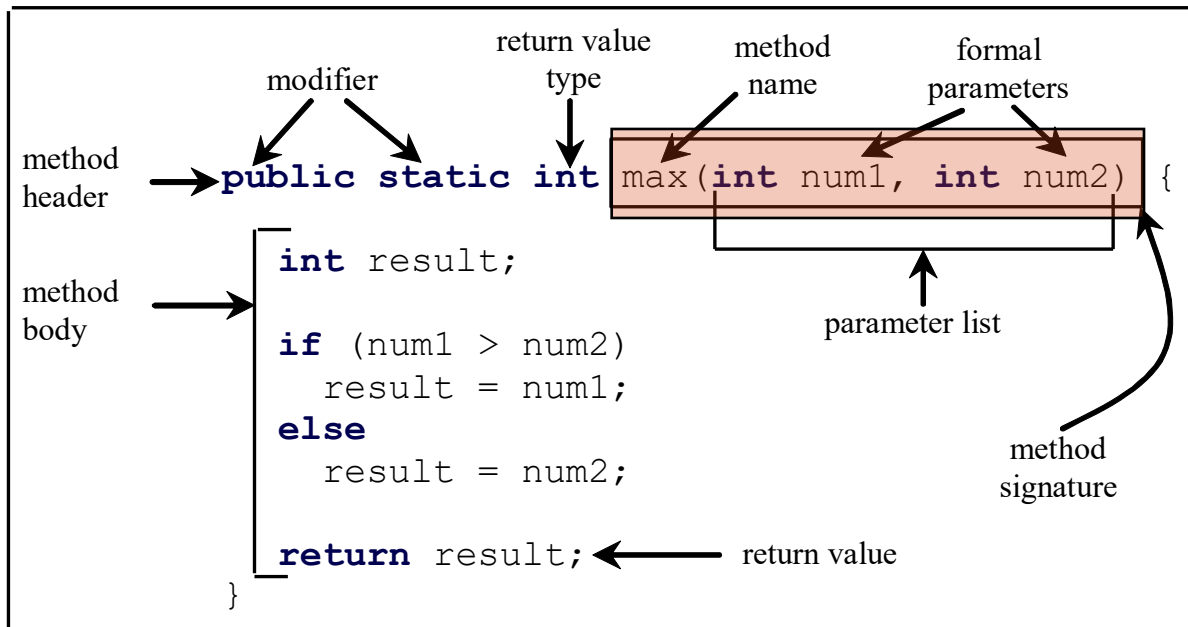
Invoke a method



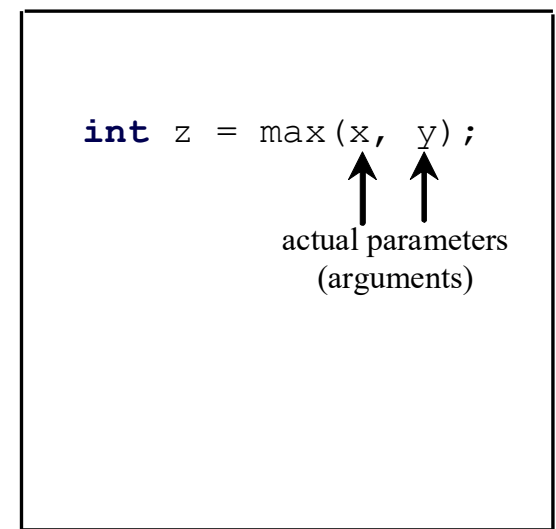
Method Signature

- Method signature* is the combination of the method name and the parameter list.

Define a method



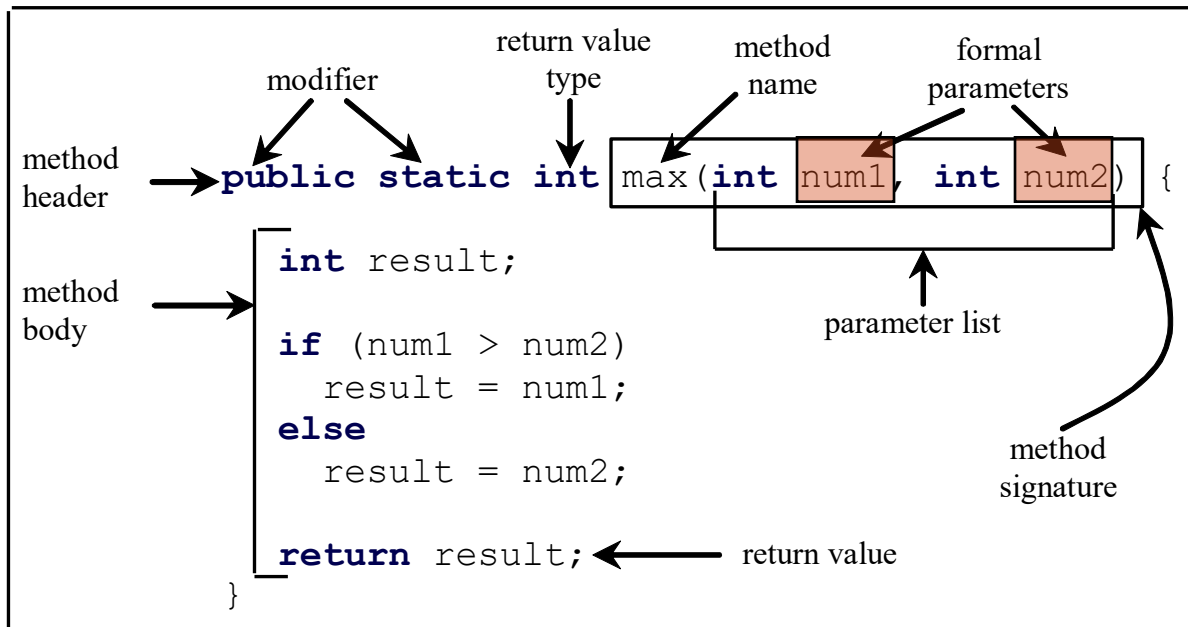
Invoke a method



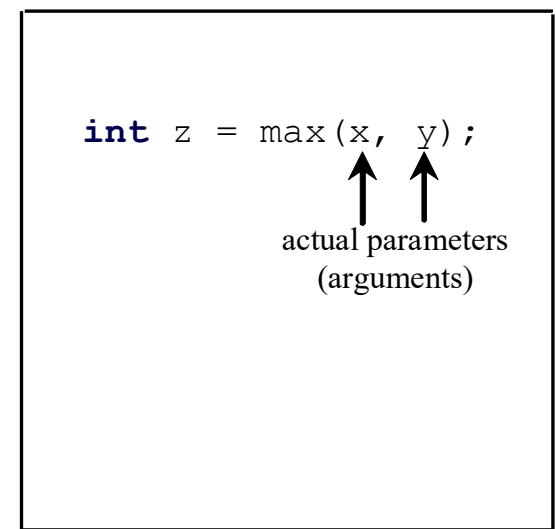
Formal Parameters

- The variables defined in the method header are known as *formal parameters*.

Define a method



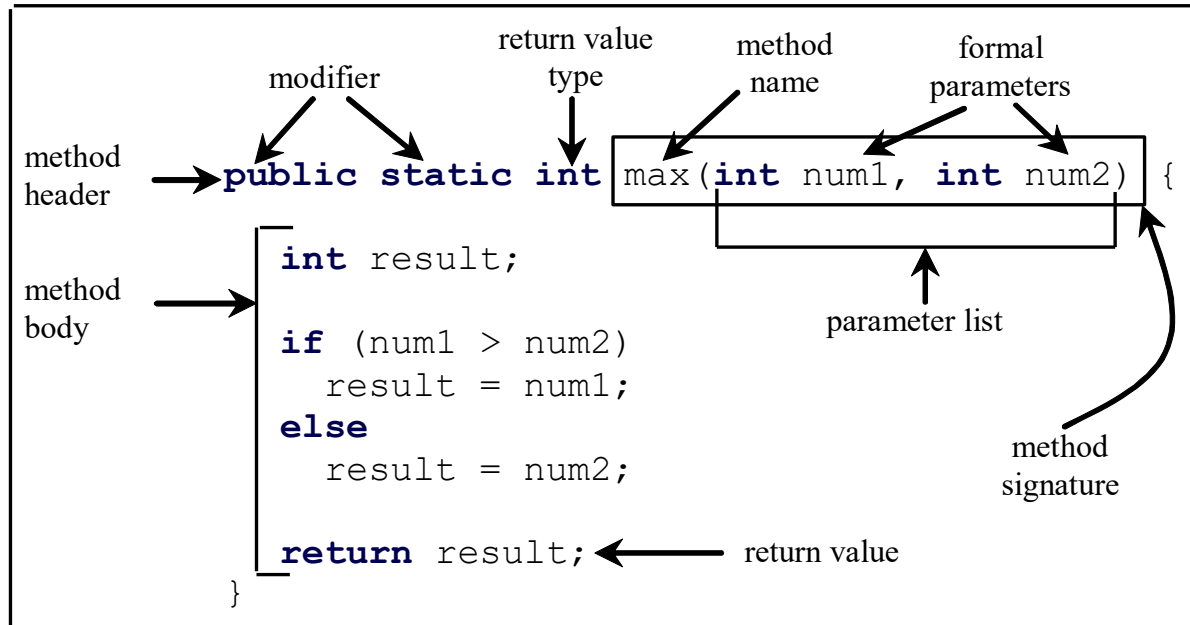
Invoke a method



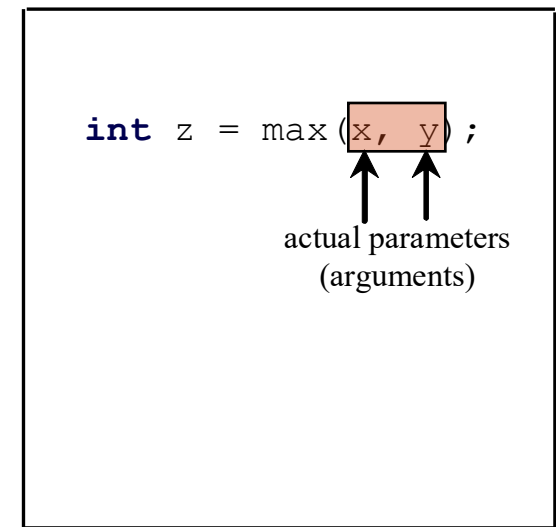
Actual Parameters

- When a method is invoked, you pass values to the formal parameter with *actual parameters* or *arguments*.

Define a method



Invoke a method

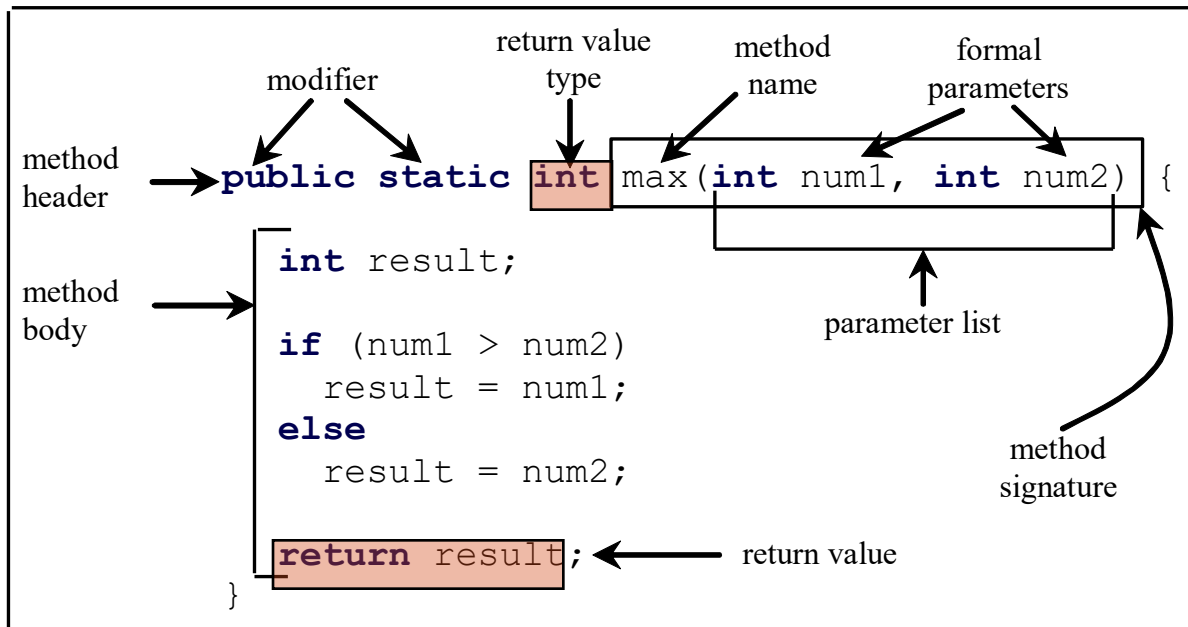


Return Value Type

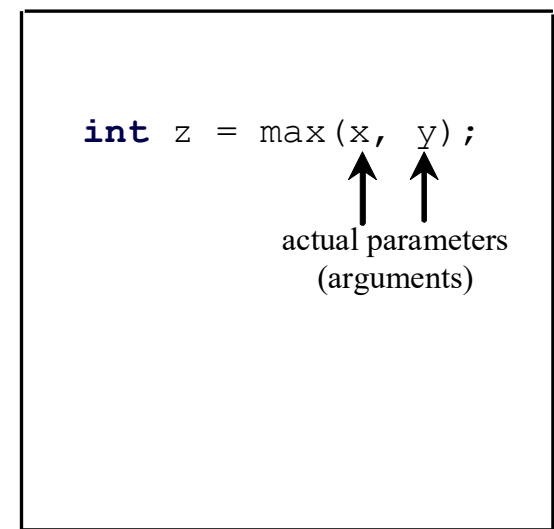
- A method may return a value

The returnValueType is the data type of the value the method returns.

Define a method

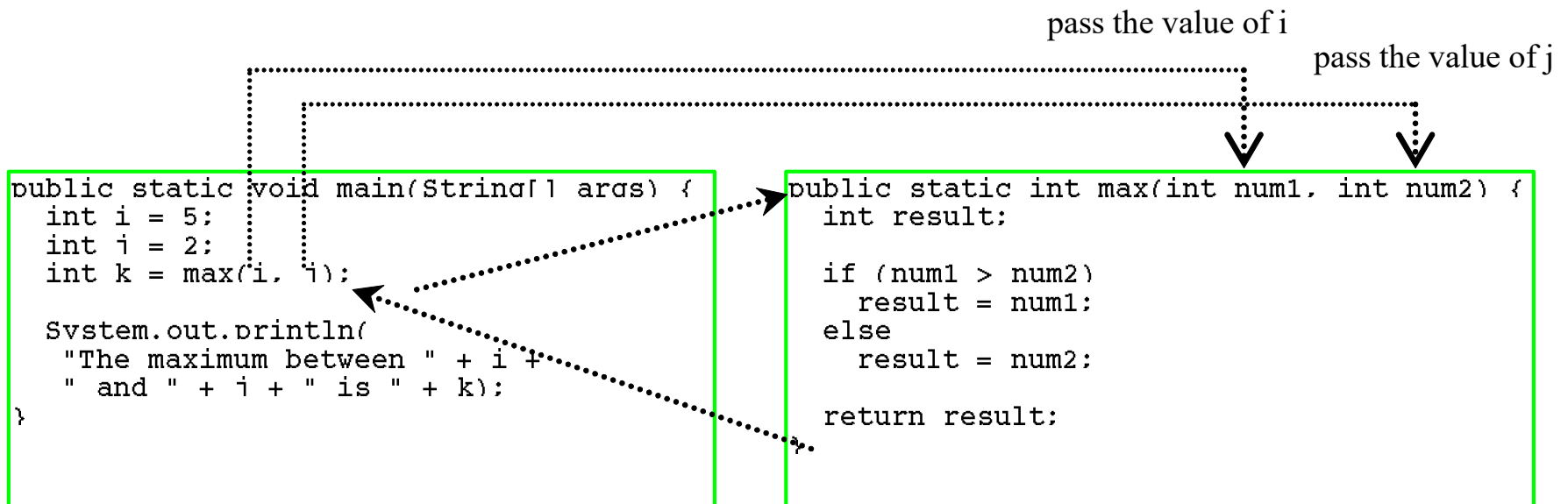


Invoke a method



If the method does not return a value, the returnValueType is the keyword void.

Calling Methods



Trace Method Invocation

i is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

j is now 2

```
public static void main(String[] args) {  
    int i = 5;  
    int i = 2;  
    int k = max(i, i);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + i + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```


Trace Method Invocation

invoke max(i, j)
Pass the value of i to num1
Pass the value of j to num2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

declare variable result

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

(num1 > num2) is true since num1 is 5 and num2 is 2

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
    return result;  
}
```

Trace Method Invocation

result is now 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

return result, which is 5

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

return max(i, j) and assign the
return value to k

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Trace Method Invocation

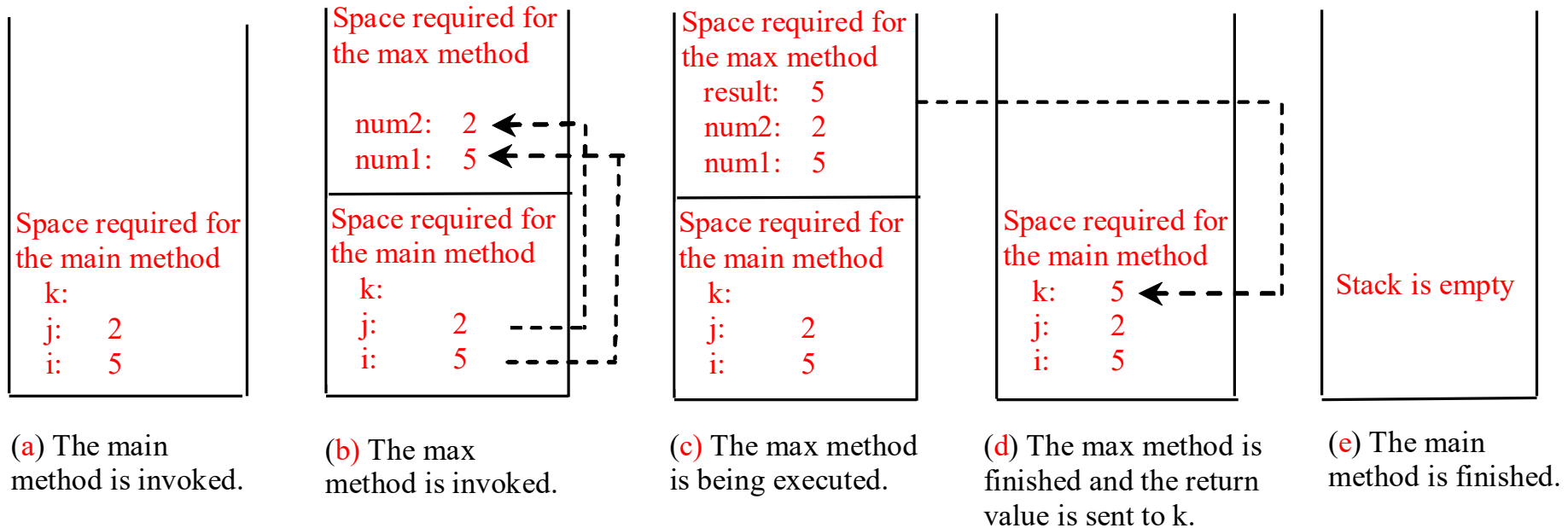
Execute the print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Call Stacks

Methods are executed using a **stack** data structure



Trace Call Stack

i is declared and initialized

```
public static void main(String[] args) {  
    int i = 5;  
    int i = 2;  
    int k = max(i, i);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + i + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

i: 5

The main method
is invoked.

Trace Call Stack

j is declared and initialized

```
public static void main(String[] args) {
    int i = 5;
    int j = 2;
    int k = max(i, i);

    System.out.println(
        "The maximum between " + i +
        " and " + i + " is " + k);
}
```

```
public static int max(int num1, int num2) {
    int result;

    if (num1 > num2)
        result = num1;
    else
        result = num2;

    return result;
}
```

j: 2
i: 5

The main method
is invoked.

Trace Call Stack

Declare k

```
public static void main(String[] args) {  
    int i = 5;  
    int i = 2;  
    int k = max(i, i);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + i + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

Trace Call Stack

Invoke max(i, j)

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

pass the values of i and j to num1
and num2

num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

pass the values of i and j to num1
and num2

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

(num1 > num2) is true

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2)  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Assign num1 to result

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2)  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Return result and assign it to k

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k: 5
j: 2
i: 5

The max method is
invoked.

Trace Call Stack

Execute print statement

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:5
j:2
i:5

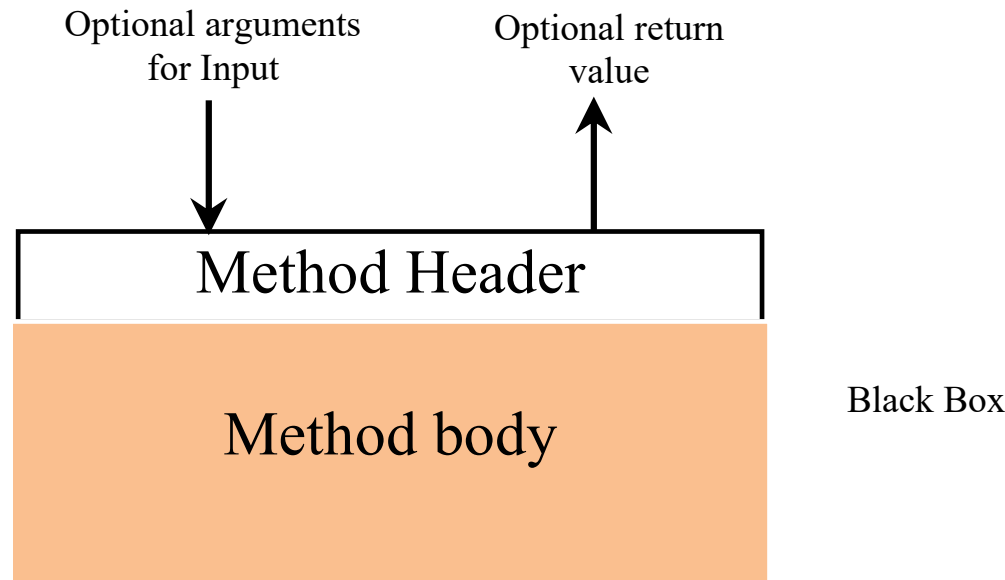
The main method
is invoked.

Benefits of Methods

1. Reuse: Write a method once and reuse it anywhere.
2. Method Abstraction and Information hiding:
 - Hide the implementation from the user.
 - Reduces complexity of the program.

Method Abstraction

Application Programming Interface (API) = the method body is a black box that contains the detailed implementation for the method.



Call-by-value

- Method formal parameters are *copies of the original data (arguments)*.
- Consequence?
 - methods cannot assign („=“) new values to primitive type formal arguments and affect the original passed variables.
- Why?
 - changing argument values changes the copy, not the original.

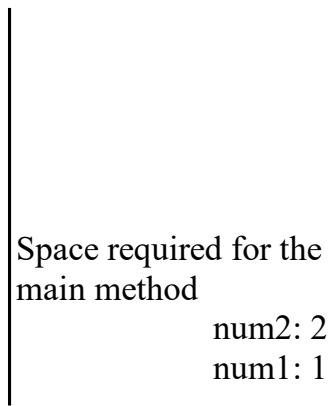
```
public class Test {  
    public static void main(String[] args) {  
        int num = 1;  
        m(num) ;  
        System.out.println(num) ; // still 1  
    }  
    public static void m(int n) {  
        n = 2;  
    }  
}
```

A method cannot swap primitive type arguments

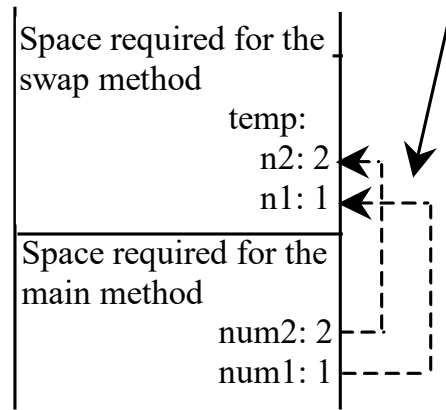
```
public class Test {  
    public static void main(String[] args) {  
        int num1 = 1;  
        int num2 = 2;  
        swap(num1, num2);  
        System.out.println(num1 + " " + num2); //still 1 2  
    }  
  
    public static void swap(int n1, int n2) {  
        int temp = n1;  
        n1 = n2;  
        n2 = temp;  
        System.out.println(n1 + " " + n2); // 2 1  
    }  
}
```

Call-by-value

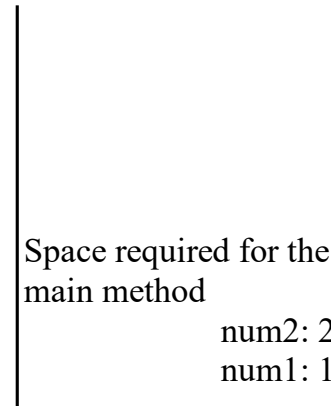
The values of num1 and num2 are passed to n1 and n2. Executing swap does not affect num1 and num2.



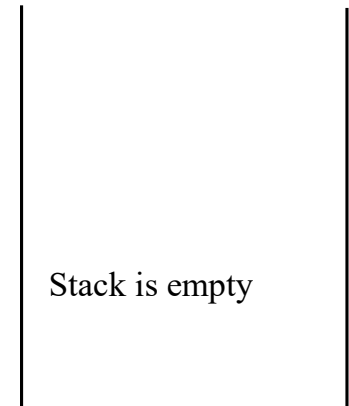
The main method is invoked



The swap method is invoked



The swap method is finished



The main method is finished

Overloading

- Method overloading is the ability to create multiple methods of the **same name** with different signatures and implementations:

```
public class Overloading {  
    public static int max(int num1, int num2) {  
        if (num1 > num2)  
            return num1;  
        return num2;  
    }  
    public static double max(double num1, double num2) {  
        if (num1 > num2)  
            return num1;  
        return num2;  
    }  
    public static void main(String[] args) {  
        System.out.println(max(1, 2)); // 2 (as an int)  
        System.out.println(max(4, 2.3)); // 4.0 (as a double)  
    }  
}
```

Overloading

- Overloaded methods must differ either by the types of their parameters or by their number of arguments)
- Method *matching* is the process to find the method implementation for the call:
 - it uses a "*best match*" algorithm to cast the actual parameters' types to the formal parameter types
 - For example:

```
System.out.println(max(1, 2)); // matches the ints method
System.out.println(max(4, 2.3)); // matches the doubles one
```

Overloading & Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match.
 - This is referred to as *ambiguous invocation*. and it is a compilation error.

Overloading & Ambiguous Invocation

```
public class AmbiguousOverloading {  
    public static double max(int num1, double num2) {  
        if (num1 > num2)  
            return num1;  
        else  
            return num2;  
    }  
    public static double max(double num1, int num2) {  
        if (num1 > num2)  
            return num1;  
        else  
            return num2;  
    }  
    public static void main(String[] args) {  
        System.out.println(max(1, 2)); // compiler error here  
        // it matches equally both methods  
    }  
}
```

CAUTION: all execution paths

- A return statement is required for a value-returning method.

The method shown below has a compilation error because the Java compiler thinks it possible that this method does not return any value if the condition is false in the last if statement.

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else if (n < 0)  
        return -1;  
}
```

(a)

Should be

```
public static int sign(int n) {  
    if (n > 0)  
        return 1;  
    else if (n == 0)  
        return 0;  
    else  
        return -1;  
}
```

(b)

To fix this problem, delete *if (n < 0)* in (a), so that the compiler will see a return statement to be reached regardless of how the if statement is evaluated.